Design Documentation

Story S4: Generate Seating Allocation

1. Introduction

Purpose:
This document presents the detailed design for Story S4: Generate Seating Allocation. The design artifacts illustrate how the system automatically allocates students to exam hall seats while enforcing 8-way anti-cheating constraints.

Design Overview:
The allocation system uses a layered architecture with:
• Presentation Layer (UI components)
• Business Logic Layer (Controllers and Algorithms)
• Data Access Layer (Database operations)

Key Design Principles:
• Separation of Concerns: Clear boundaries between layers
• Constraint Enforcement: 8-way adjacency checks prevent cheating
• Scalability: Algorithm designed for 1000+ students
• Flexibility: Support for irregular hall shapes

2. Sequence Diagram - Generate Allocation

Overview:
The sequence diagram shows the temporal flow of interactions when an admin generates a seating allocation. It illustrates message passing between objects in chronological order.

Participating Objects:
• Admin (Actor) - Initiates the allocation process
• AllocationUI (Boundary) - User interface component
• AllocationController (Control) - Orchestrates the allocation workflow
• AllocationAlgorithm (Control) - Implements seat assignment logic
• ConstraintValidator (Control) - Validates constraints
• Database (Entity) - Persistent data storage

Message Flow:
1. Admin clicks "Generate Allocation" button
2. UI sends generateAllocation(examId, mode) to Controller
3. Controller retrieves halls from Database
4. Controller retrieves students for exam from Database
5. Controller requests capacity validation from Validator
6. Controller delegates allocation to Algorithm
7. Algorithm performs seat assignment with 8-way checks (loop)
8. Controller saves allocations to Database
9. UI displays results to Admin

Design Rationale:
The sequence shows clear separation of responsibilities. The Controller coordinates without implementing business logic, while the Algorithm focuses purely on seat assignment.

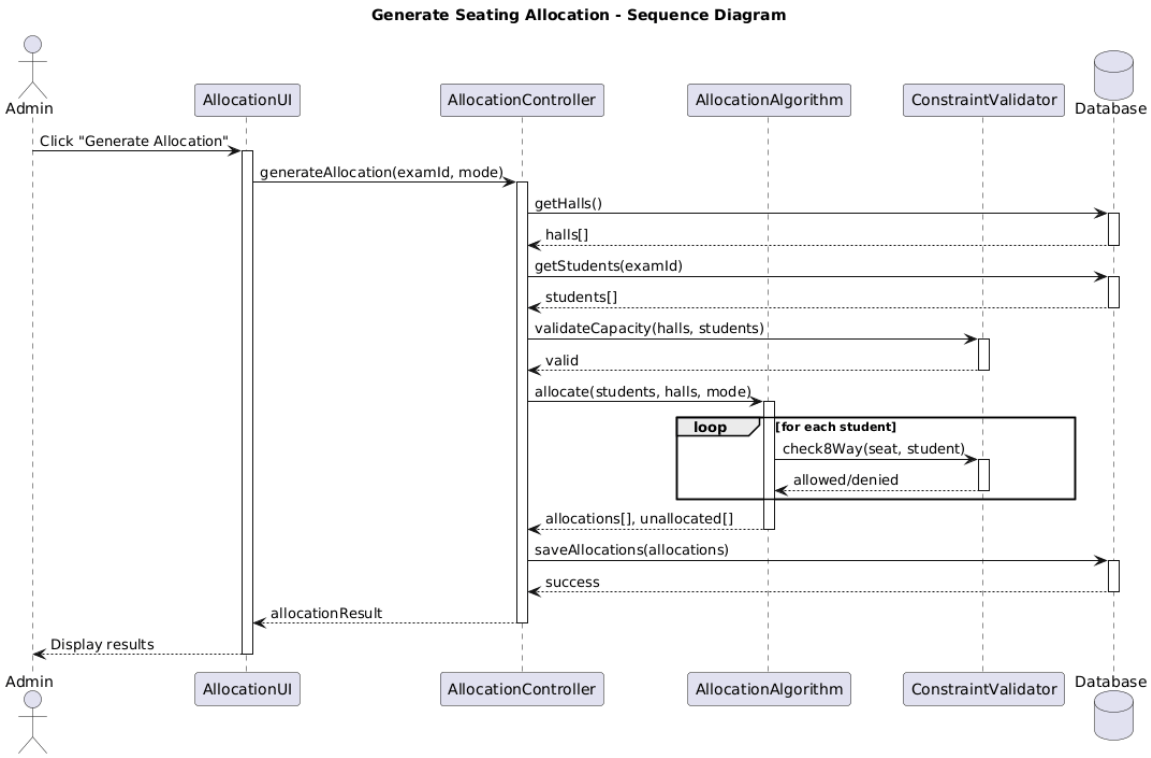

Figure 1: Sequence Diagram - Generate Allocation Flow

3. Collaboration Diagram - 8-Way Constraint Check

Overview:

The collaboration diagram shows object relationships for 8-way constraint validation.

Message Numbering:

1: check8Way(targetSeat, studentPaper) - Algorithm → Validator
2: getAdjacentSeats(row, col) - Validator → Seat
3: adjacentSeats[8] - Seat → Validator
4.1: getAllocation(seatId) - Validator → SeatAllocation [loop]
4.2: allocation(student, paper) - SeatAllocation → Validator
4.3: if paper == studentPaper return CONFLICT
5: return ALLOWED/CONFLICT - Validator → Algorithm

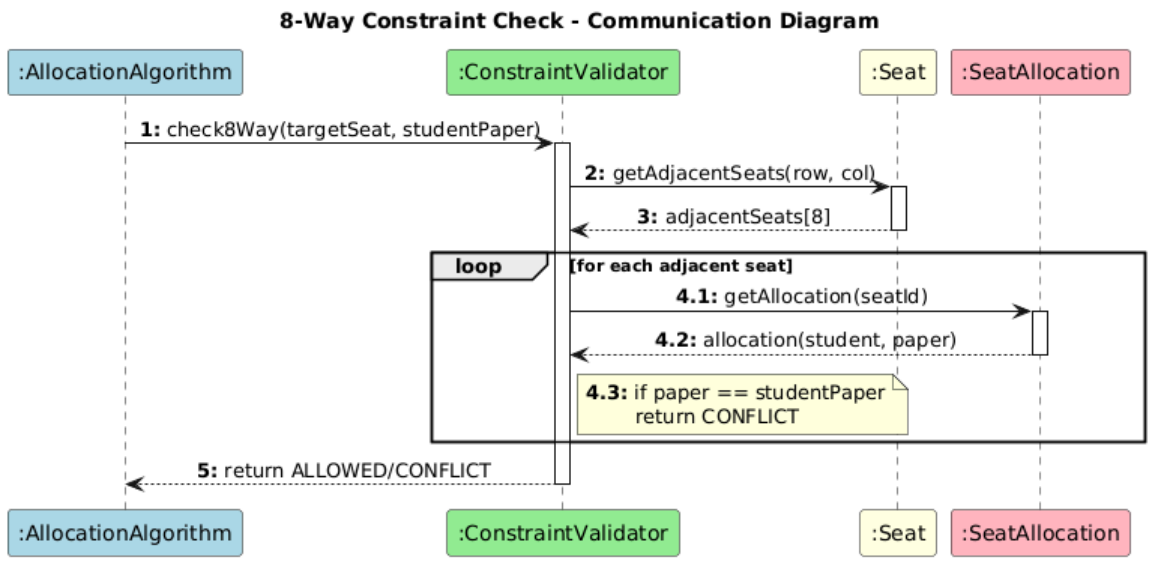

Figure 2: Collaboration Diagram - 8-Way Constraint

4. Activity Diagram - Allocation Algorithm

Algorithm Phases:

Phase 1: Load students and halls
Phase 2: For each hall, calculate capacity and create grid
Phase 3: For each student, find seat, check constraints, assign
Phase 4: Return allocation results

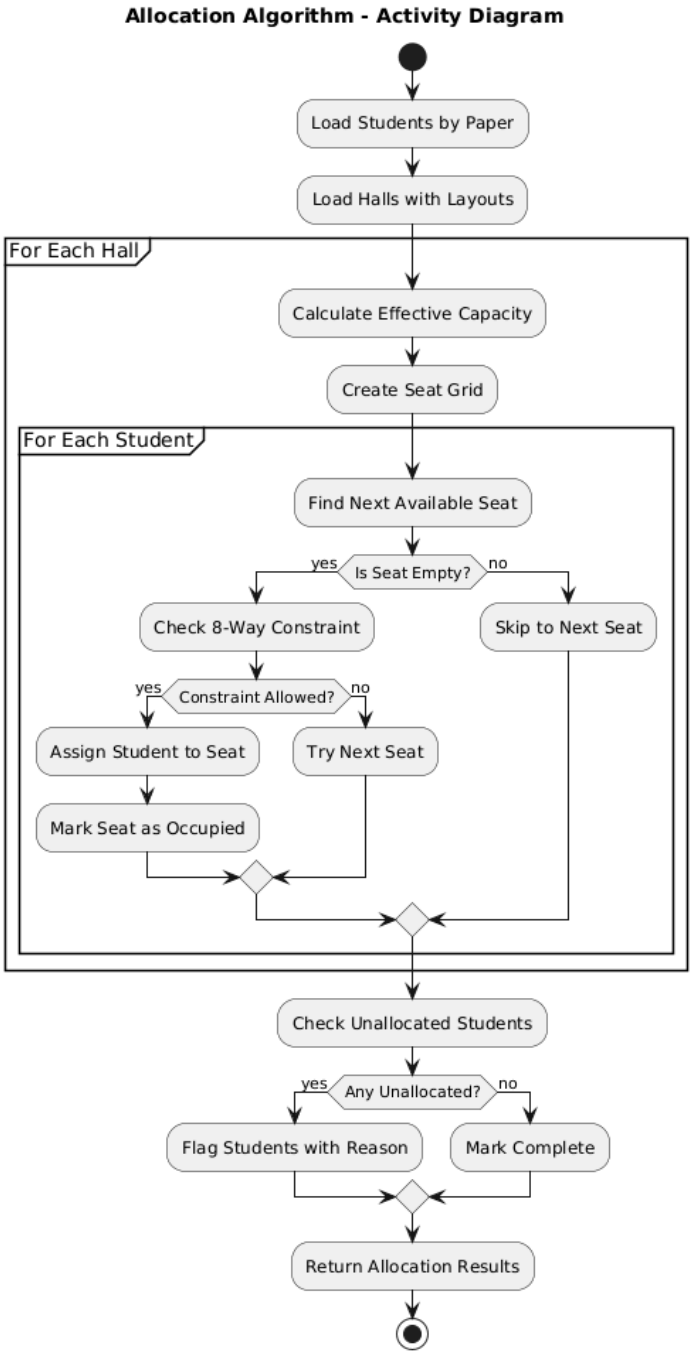

Figure 3: Activity Diagram - Algorithm Logic

5. Class Diagram - System Architecture

Entity Classes:
Student, Hall, Seat, SeatAllocation, Exam

Control Classes:
AllocationController, AllocationAlgorithm, ConstraintValidator

Key Relationships:
• Hall 1 *---> many Seat (Composition)
• SeatAllocation ---> Student, Hall, Seat, Exam (Association)
• Controller uses Algorithm uses Validator (Dependency)

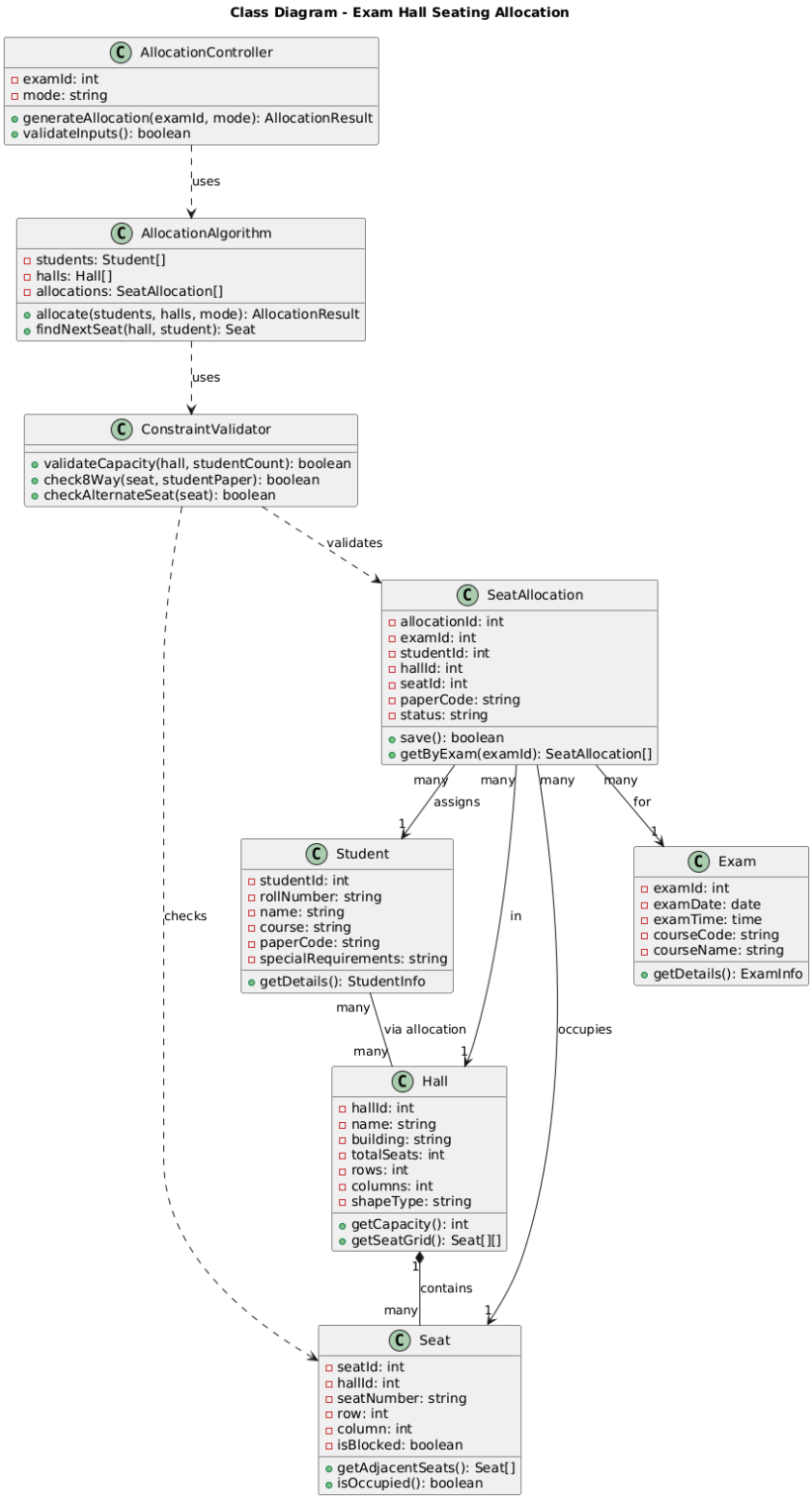

Figure 4: Class Diagram - Complete Architecture

6. Design Summary

Design Coverage:
This documentation provides UML views covering temporal flow (sequence), object collaboration (collaboration), workflow logic (activity), and static structure (class).

Implementation Ready:
Sprint 1 (by 3 March 2026): Core allocation with 8-way checking
Sprint 2 (by 17 March 2026): Reports and notifications

Designed By: Abhishek Roy, Sahil Siddique
Date: 17-02-2026